

Exercise Sheet 5: Training and Diagnosing Conditional GANs

Due on 05.06.2026, 10:00

Tommaso Martorella and Pingchuan Ma (Tommaso.Martorella@lmu.de /
P.Ma@lmu.de)**Important Notes:**

1. **Two-week assignment:** This exercise sheet is worth **40 points** and is scheduled over two weeks. Plan training runs early – GANs are stochastic and may require several attempts.
2. **Questions:** Please use the Moodle platform and post your questions to the forum. They will be answered by us or your fellow students.
3. **Submission:** Your submission should consist of one single ZIP file. Both the PDF file and the ZIP file should contain your surname and your matriculation number (**Surname-MatriculationNumber.zip**) for grading purposes. Please submit the completed .ipynb notebook (the provided template, with your code and answers filled in) together with an exported PDF version of the notebook serving as the report. **Submissions that fail to follow the naming convention or miss the PDF/notebook files will not be graded.**
4. **Implementation:** Use Python and PyTorch. Use relative paths only. Non-trivial code sections should be commented, and plots must have informative axis labels, legends, and captions.
5. **Deadline:** The due date for this exercise is **05.06.2026, 10:00**.

Task 1: Optimal Discriminator and Generator (8P)

For GANs, the minimax game objective for adversarial learning was defined as follows:

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]. \quad (1)$$

Equivalently, by introducing the generator density $p_G(x)$ induced by sampling $z \sim p(z)$ and pushing through G , we can write the objective as an integral over x :

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx. \quad (2)$$

The goal of this task is to derive the optimal discriminator and generator from scratch and to discuss the practical generator loss used during training.

- a) **Optimal discriminator (3P).** Using (2), derive the optimal discriminator $D^*(x)$ for a fixed generator G in terms of $p_{\text{data}}(x)$ and $p_G(x)$.

Hint: For fixed $a, b > 0$, find the maximum of $f(y) = a \log y + b \log(1 - y)$ over $y \in (0, 1)$ by setting $f'(y) = 0$.

- b) **Generator optimum and equilibrium (3P).** Substitute D^* into (2) and discuss the global minimum of $V(D^*, G)$ over G . In particular, argue at which point this minimum is reached, what value D^* takes there, and what this means in terms of distinguishing real and generated samples. You may briefly mention the connection to the Jensen-Shannon divergence $\text{JSD}(p_{\text{data}} \parallel p_G)$, but a full proof is not required.

- c) **Non-saturating generator loss (2P).** The saturating generator loss derived from (1) is

$$\mathcal{L}_G^{\text{sat}} = \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))],$$

whereas in practice GANs are typically trained with the non-saturating generator loss

$$\mathcal{L}_G^{\text{ns}} = -\mathbb{E}_{z \sim p(z)} [\log D(G(z))].$$

Explain why $\mathcal{L}_G^{\text{ns}}$ is preferred in practice. Your answer should discuss the gradient signal early in training (when $D(G(z)) \approx 0$) and clarify that the optimum is unchanged, but optimization is improved.

Please include all necessary steps to get to the final results in your report.

Note: If you are stuck, take a look at the seminal paper *Generative Adversarial Nets* by Goodfellow et al.¹.

¹<https://arxiv.org/pdf/1406.2661>

Task 2: Training a Conditional DCGAN on Fashion-MNIST (14P)

In this task you will implement a class-conditional Generative Adversarial Network (GAN) from scratch and train it on the Fashion-MNIST dataset.² The generator should receive a 100-dimensional standard normal latent code $z \sim \mathcal{N}(0, I)$ together with a class label $y \in \{0, \dots, 9\}$ and produce a 32×32 grayscale image. Use the DCGAN³ architecture shown in Figure 1 as a starting point. The discriminator receives both the image and the class label and outputs a real/fake logit. Use (1) as the theoretical training signal, but for the generator use the **non-saturating** loss from Task 1c) in practice.

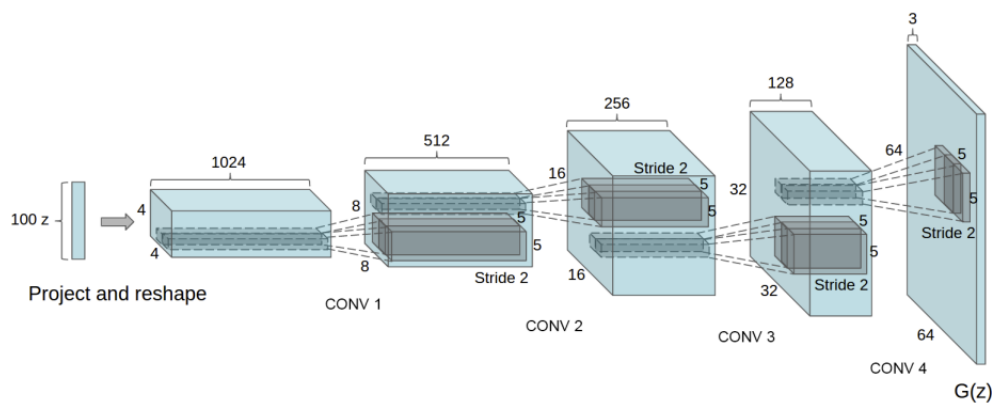


Figure 1: The DCGAN generator architecture. The latent code is linearly projected and reshaped to a 4×4 grid. Afterwards, *torch.nn.Upsample* are applied for upsampling. After every convolution layer, there is a BatchNorm layer followed by a ReLU activation. A similar but reversed architecture (using strided convolutions and BatchNorm/LeakyReLU) can be used for the discriminator.

Class conditioning. Both networks must be conditioned on the class label. Acceptable conditioning mechanisms include (but are not limited to):

- label embedding concatenated to the latent vector (generator) and to the image as a spatial map (discriminator);
- projection conditioning / conditional BatchNorm;
- any other clearly explained conditioning method.

²<https://pytorch.org/vision/stable/generated/torchvision.datasets.FashionMNIST.html>

³<https://arxiv.org/pdf/1511.06434>

Clearly state in your report which mechanism you chose.

Training loss. Train the discriminator with the standard binary cross-entropy loss

$$\mathcal{L}_D = -\mathbb{E}_{x \sim p_{\text{data}}}[\log D(x, y)] - \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z, y), y))],$$

and the generator with the non-saturating loss

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p(z)}[\log D(G(z, y), y)].$$

Use `torch.nn.BCEWithLogitsLoss` for numerical stability.

Required report items. Your final submitted notebook must include:

- A figure containing the evolution of training losses for the discriminator and the generator.
- A figure containing the classification accuracy of the discriminator on real and on fake samples per epoch.
- A grid of randomly synthesised images from your final generator.
- Fixed-latent evolution: pick a single fixed latent code z and class label y , and show the generated image $G(z, y)$ at several training checkpoints.
- A t-SNE⁴ embedding of images from the train split, the test split, and your generator for at least 5 classes of your choice, with around 500 images per source. For the embedding, train an extra CNN classifier on real Fashion-MNIST data, freeze it, and use features from an intermediate layer (suggested: the layer before the classification head).
- The classification accuracy on generated data, using the classifier from the previous bullet point, with respect to the conditioning labels.
- Your Python code.

plotting code is already provided by us for all of the above.

Hints:

- Use the Adam optimiser with $\text{lr} = 10^{-4}$ and $\beta_1 = 0.2$, $\beta_2 = 0.95$.
- Fashion-MNIST has a standard resolution of 28×28 , so resize to 32×32 and normalise to $[-1, 1]$ (use `Tanh` as final activation in the generator).

⁴<https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>

- You may need to adapt the DCGAN architecture to produce 32×32 images instead of 64×64 .
- You should already see decent results after only a few epochs.
- With the baseline implementation, you can reasonably expect the loss to **diverge**.

Point breakdown:

Subtask	Points
Correct conditional generator and discriminator implementation	4
Correct GAN training loop with non-saturating generator loss	3
Loss and discriminator-accuracy plots	2
Final generated samples and fixed-latent evolution	2
Classifier training and t-SNE visualisation	2
Generated-data classifier accuracy and code quality	1
Total	14

Task 3: GAN Stabilization Study (10P)

GAN training is notoriously unstable: discriminator explosions, mode collapse, and vanishing generator gradients are common. In this task you will compare your baseline conditional DCGAN from Task 2 with **three (one given)** additional stabilisation variants of your choice, trained on the same dataset under otherwise identical hyperparameters.

Available variants:

- A) **Spectral normalisation.** Wrap the discriminator convolutional and linear layers with `torch.nn.utils.parametrizations.spectral_norm`. Constrains the Lipschitz constant of D .
- B) **One-sided label smoothing.** Replace the real labels by e.g. 0.9 (do not smooth fake labels unless you explicitly justify it). Prevents the discriminator from becoming overconfident.
- C) **R1 gradient penalty.** Add the penalty

$$\mathcal{L}_{R_1} = \frac{\gamma}{2} \mathbb{E}_{x \sim p_{\text{data}}} [\|\nabla_x D(x, y)\|_2^2]$$

to the discriminator loss (typical $\gamma \in [1, 10]$). Discourages overly sharp discriminator gradients on real samples.

Required report items. For the baseline and for each of your two chosen variants, provide:

- generator and discriminator loss curves;
- discriminator real/fake accuracy curves;
- a grid of generated samples from the final model;
- the classifier accuracy on generated samples (using the Task 2 classifier);
- a t-SNE embedding of train/test/generated images (same procedure as in Task 2);
- a small comparison table summarising the above.

plotting code is already provided by us for all of the above.

In addition, write a short discussion for *each* variant answering some of the following questions:

1. What problem is this method supposed to address?

2. How does the method modify the training objective, architecture, or data?
3. Did it improve stability in your run?
4. Did it improve generated sample quality?
5. Did it improve class consistency?
6. Did it introduce any downside (slower training, worse samples, weaker discriminator, implementation complexity)?

Discussions must be grounded in your own plots and samples, not in generic textbook statements.

Point breakdown:

Subtask	Points
Correct implementation of two stabilisation variants	4
Loss and discriminator-accuracy comparison against baseline	1
Generated sample comparison against baseline	1
Generated-data classifier accuracy comparison	1
Per-variant t-SNE comparison against baseline	1
Discussion of variant 1	1
Discussion of variant 2	1
Total	10

Task 4: Conditional Control and Interpolation (4P)

This task evaluates whether your generator actually uses both the latent code z and the class label y . Use your best-performing model from Task 2 or Task 3.

- Fixed latent, varying class label (1P).** Pick a single fixed latent code z and generate $G(z, y)$ for all $y \in \{0, \dots, 9\}$. Produce one row of 10 generated images, one column per Fashion-MNIST class.
- Spherical latent interpolation (1P).** Fix one class label y , draw two latent codes $z_1, z_2 \sim \mathcal{N}(0, I)$, and generate images for the spherical interpolation

$$z_\alpha = \frac{\sin((1-\alpha)\omega)}{\sin \omega} z_1 + \frac{\sin(\alpha\omega)}{\sin \omega} z_2, \quad \omega = \arccos(\langle \hat{z}_1, \hat{z}_2 \rangle),$$

with $\hat{z} = z/\|z\|$ and $\alpha \in \{0.0, 0.1, \dots, 1.0\}$. Slerp is *variance preserving*: $\|z_\alpha\| \approx \|z_1\| \approx \|z_2\|$ for every α , whereas plain linear interpolation $(1-\alpha)z_1 + \alpha z_2$ shrinks the midpoint by a factor of about $1/\sqrt{2}$ and pulls samples off the typical $\mathcal{N}(0, I)$ shell the generator was trained on. Repeat for at least one class (preferably two) and arrange the resulting images in α -order.

Code for the spherical interpolation is provided.

- Class-to-class interpolation (1P).** Fix one latent code z and pick two classes y_1, y_2 . *Linearly* interpolate between their class embeddings,

$$e_\alpha = (1-\alpha) G_{\text{embed}}(y_1) + \alpha G_{\text{embed}}(y_2), \quad \alpha \in \{0.0, 0.1, \dots, 1.0\},$$

and push (z, e_α) through the rest of the generator (you will need to bypass `G.forward` and call `G.embed` directly so you can feed an arbitrary embedding instead of an integer label). Linear interpolation is the right choice here because the embedding space is learned end-to-end and is *not* constrained to any sphere, so the variance-preservation argument from b) does not apply. Repeat for at least two class pairs.

- Discussion (1P).** Briefly answer:

- Does changing y while fixing z change the semantic class (Task 4a)?
- Is the spherical z -interpolation (Task 4b) smooth, and do the endpoints look like distinct samples of the same class?
- Are the class-to-class morphs (Task 4c) smooth, or do you observe abrupt transitions?
- Does the generator appear to ignore z or y ?

- Which classes are easiest/hardest to control?

Task 5: Confusion Matrix on Generated Data (4P)

The final task quantifies whether your conditional generator actually respects the conditioning labels.

Using the Fashion-MNIST classifier trained in Task 2:

1. Generate a **balanced** set of class-conditional samples (200 samples per class, 10 classes, for a total of 2,000 images).
2. For each generated image record the conditioning label y_{cond} and the classifier prediction $\hat{y}$.
3. Compute the confusion matrix

$$C_{ij} = \#\{\text{generated samples with } y_{\text{cond}} = i \text{ classified as } j\},$$

where rows correspond to conditioning labels and columns to classifier predictions.

4. Report the per-class generated accuracy

$$\text{acc}_i = \frac{C_{ii}}{\sum_j C_{ij}}$$

as a table or bar plot.

5. Discuss the most common confusions. Typical examples to look for include T-shirt/top vs. shirt, pullover vs. coat, and sandal vs. sneaker. Comment on whether the generator faithfully follows the conditioning labels.

plotting code is already provided by us for all of the above.

Point breakdown:

Subtask	Points
Generate a balanced class-conditional sample set	1
Compute classifier predictions on generated samples	1
Report the confusion matrix	1
Discuss per-class accuracy and common confusions	1
Total	4